



Manual Técnico Integral - Sistema Edubboks

Documentación Técnica Completa para Equipos de Desarrollo, QA y DevOps



Tabla de Contenidos

1. [Introducción](#) 1.1. [Propósito del Documento](#) 1.2. [Funcionalidades Principales \(Requerimientos Funcionales\)](#)
 2. [Arquitectura General del Sistema](#) 2.1. [Stack Tecnológico](#) 2.2. [Diagrama de Arquitectura de Alto Nivel](#) 2.3. [Diagrama de Despliegue](#)
 3. [Tecnologías y Dependencias Críticas](#) 3.1. [Backend \(API\)](#) 3.2. [Frontend \(App Móvil\)](#) 3.3. [Servicios Externos](#)
 4. [Arquitectura del Backend \(edubboks-api\)](#) 4.1. [Estructura de Carpetas](#) 4.2. [Modelos de Datos y Base de Datos](#) 4.3. [Autenticación y Seguridad](#)
 5. [Arquitectura del Frontend \(edubboks-app\)](#) 5.1. [Estructura de Carpetas](#) 5.2. [Flujo de Datos y Servicios](#)
 6. [API REST - Endpoints](#) 6.1. [Módulos de la API](#) 6.2. [Ejemplo de Endpoint Detallado](#)
 7. [Infraestructura y Despliegue](#) 7.1. [Entornos de Despliegue](#) 7.2. [Configuración de Render \(Backend\)](#) 7.3. [Variables de Entorno Críticas](#)
 8. [Instalación y Configuración Local](#) 8.1. [Requisitos Previos](#) 8.2. [Instalación del Backend](#) 8.3. [Instalación del Frontend](#)
 9. [Mantenimiento, Logs y Seguridad](#) 9.1. [Gestión de Errores y Logs](#) 9.2. [Consideraciones de Seguridad](#) 9.3. [Requerimientos para Mantenimiento Futuro](#)
-

1. Introducción

1.1. Propósito del Documento

El presente **Manual Técnico Integral** tiene como objetivo principal servir de referencia detallada y profesional para los equipos de **Desarrollo**, **QA** (Aseguramiento de Calidad) y **DevOps** involucrados en el mantenimiento, evolución y despliegue del sistema **Edubboks**.

Este documento describe la arquitectura, las tecnologías, la estructura del código, los modelos de datos y los procedimientos de instalación y despliegue, asegurando una comprensión profunda del sistema para facilitar:

- La incorporación de nuevos miembros al equipo.

- La resolución eficiente de *bugs* y problemas en producción.
- La planificación de nuevas funcionalidades y la escalabilidad del sistema.

1.2. Funcionalidades Principales (Requerimientos Funcionales)

El sistema **Edubboks** es una plataforma de gestión de biblioteca digital diseñada para entornos educativos, con las siguientes funcionalidades clave:

Módulo	Funcionalidad Principal	Descripción
Autenticación	Registro y Acceso	Permite a los usuarios (Estudiantes, Docentes, Administradores) autenticarse utilizando un sistema híbrido basado en Supabase Auth y Django JWT .
Catálogo	Búsqueda y Gestión de Libros	Permite a los usuarios buscar libros en el catálogo interno y realizar búsquedas externas a través de la Google Books API . Los administradores pueden gestionar el inventario.
Préstamos	Solicitud y Gestión de Préstamos	Los usuarios pueden solicitar préstamos de libros. El sistema gestiona el estado del libro (Disponible, Prestado, Reservado) y aplica límites de préstamos activos por usuario.
Sanciones	Control de Devoluciones y Multas	El sistema registra y gestiona las sanciones por devoluciones tardías, incluyendo el cálculo de multas y la inhabilitación temporal del usuario.
Administración	Gestión de Usuarios y Reportes	Los administradores tienen acceso a <i>endpoints</i> específicos para gestionar

Módulo	Funcionalidad Principal	Descripción
		usuarios, aprobar/rechazar préstamos y generar estadísticas de uso.

2. Arquitectura General del Sistema

El sistema Edubboks sigue una arquitectura de **microservicios ligeros** y **separación de responsabilidades**, implementado como un *monorepo* que aloja dos componentes principales: el *Backend* (API) y el *Frontend* (App Móvil).

2.1. Stack Tecnológico

El sistema se basa en un *stack* moderno y robusto, diseñado para la escalabilidad y el desarrollo rápido.

Componente	Tecnología Principal	Versión Clave	Propósito
Backend (API)	Python / Django	Django==5.2.0	Lógica de negocio, gestión de datos y exposición de la API REST.
API Framework	Django REST Framework	djangorestframework==3.15.2	Creación de <i>endpoints</i> RESTful.
Frontend (App)	Angular / Ionic	Angular^20.0.0 / Ionic^8.0.0	Interfaz de usuario móvil multiplataforma (iOS, Android, Web).
Contenedor Nativo	Capacitor	Capacitor^7.4.4	Puente para acceder a funcionalidades nativas del dispositivo.
Base de Datos	PostgreSQL	PostgreSQL 15.x	Almacenamiento de datos relacionales.

Componente	Tecnología Principal	Versión Clave	Propósito
Servicios Cloud	Supabase	N/A	Proveedor de servicios de autenticación (Auth) y base de datos (PostgreSQL).
Servidor Web	Gunicorn	gunicorn==23.0.0	Servidor WSGI para el despliegue de Django en producción.

2.2. Diagrama de Arquitectura de Alto Nivel

El sistema opera bajo un modelo de **Arquitectura de Tres Capas** con una clara división entre la capa de presentación (Frontend), la capa de aplicación (Backend) y la capa de datos (Supabase).

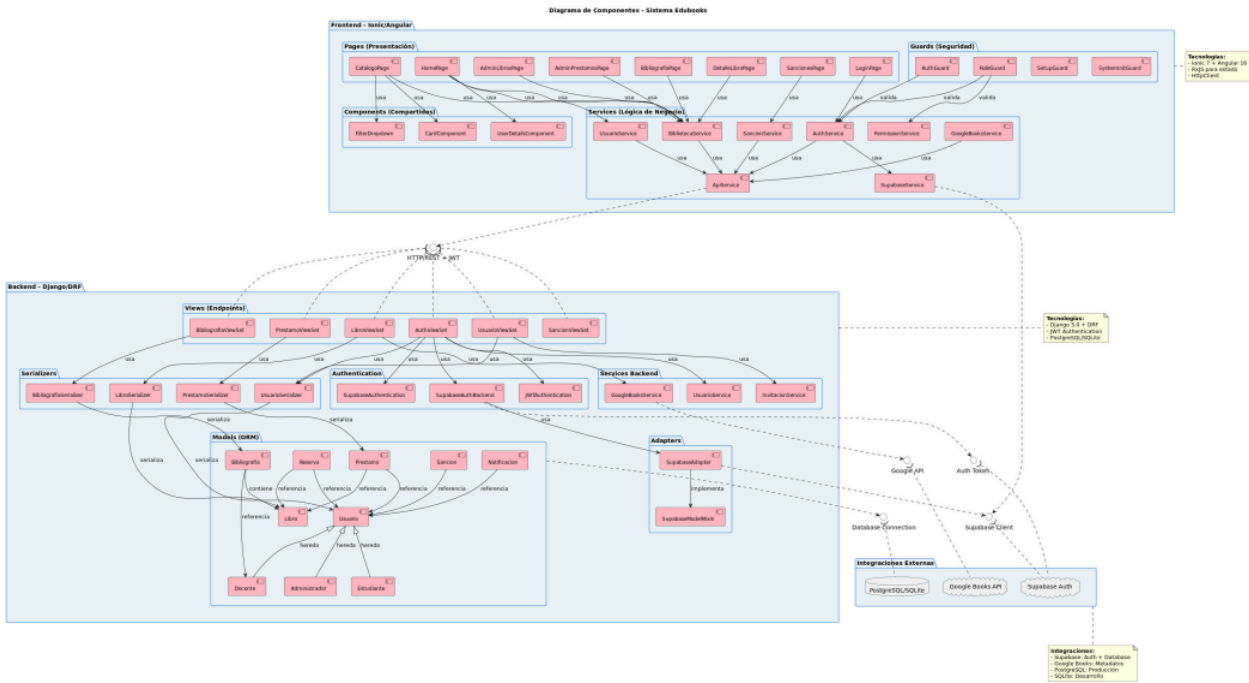
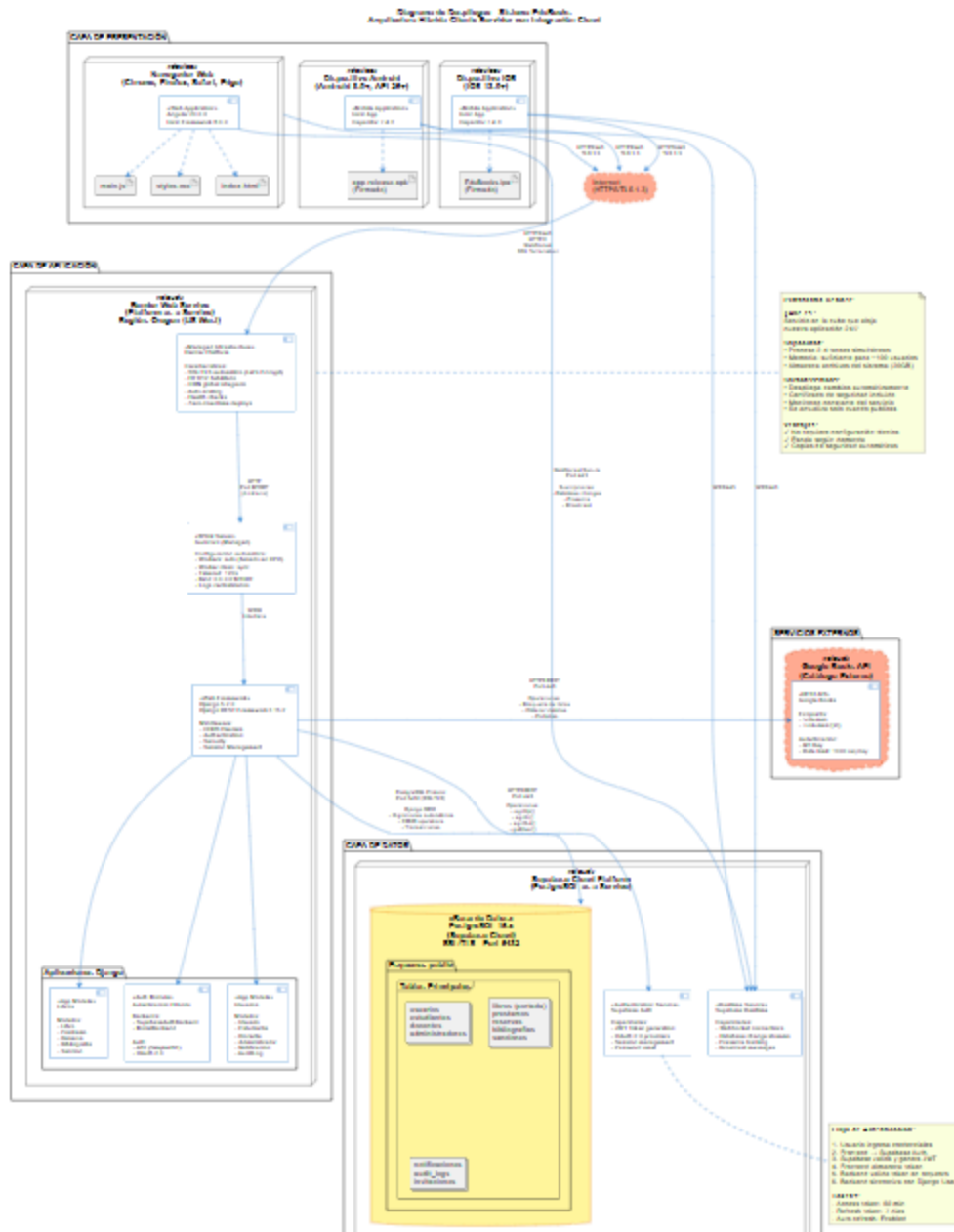


Diagrama de Componentes - Edubooks.svg

2.3. Diagrama de Despliegue

El diagrama de despliegue ilustra la distribución física de los componentes del sistema en los entornos de producción.



3. Tecnologías y Dependencias Críticas

3.1. Backend (API)

El *backend* está construido sobre **Python 3.11+** y el *framework* **Django**.

Dependencia	Versión	Propósito
Django	5.2.0	Framework principal de desarrollo web.
djangorestframework	3.15.2	Creación de la API REST.
djangorestframework-simplejwt	5.3.0	Manejo de <i>tokens</i> JWT para la autenticación.
supabase	N/A	Cliente Python para interactuar con los servicios de Supabase.
psycopg2-binary	2.9.11	Adaptador de PostgreSQL para Django.
python-decouple	3.8	Gestión de variables de entorno (<i>.env</i>).
gunicorn	23.0.0	Servidor WSGI utilizado en producción (Render).
requests	2.32.3	Cliente HTTP para la integración con Google Books API.

3.2. Frontend (App Móvil)

El *frontend* es una aplicación móvil híbrida construida con **Angular** e **Ionic**, empaquetada con **Capacitor**.

Dependencia	Versión	Propósito
@angular/core	^20.0.0	Core del <i>framework</i> Angular.

Dependencia	Versión	Propósito
@ionic/angular	^8.0.0	Componentes de UI y navegación para aplicaciones móviles.
@capacitor/core	^7.4.3	Core de Capacitor para la abstracción de la plataforma nativa.
@capacitor/android	^7.4.4	Implementación de Capacitor para Android.
@supabase/supabase-js	N/A	Cliente JavaScript para interactuar con Supabase (Auth y Realtime).
rxjs	N/A	Programación reactiva para la gestión de estados y flujos de datos.

3.3. Servicios Externos

- **Supabase:** Utilizado como proveedor de **PostgreSQL** y como sistema de **Autenticación (Auth)**. El *backend* de Django utiliza un sistema híbrido donde los usuarios se crean en Supabase y se sincronizan con el modelo **Usuario** de Django.
- **Google Books API:** Utilizado por el módulo **libros** para buscar metadatos de libros (título, autor, ISBN, portada) que no se encuentran en el inventario local.

4. Arquitectura del Backend (edubooks-api)

El *backend* sigue la estructura de un proyecto Django modularizado, con aplicaciones separadas para las funcionalidades principales.

4.1. Estructura de Carpetas

El proyecto se organiza en un *monorepo* con el *backend* ubicado en **edubooks-api/**.

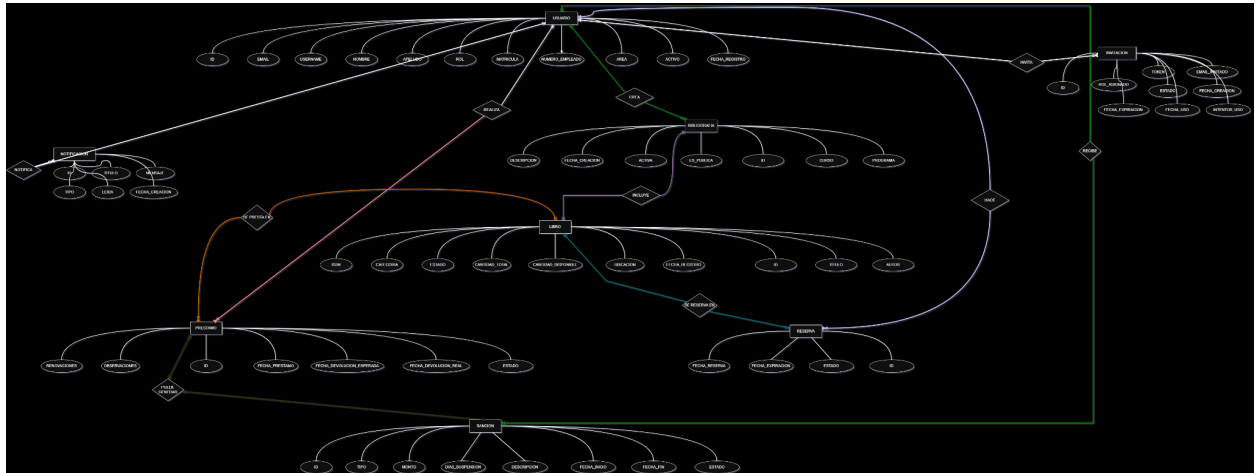
```
edubooks-api/
├── edubooks/           # Configuración del proyecto
│   └── settings.py     # Configuración Django
```

```
|   ├── urls.py          # URLs principales
|   ├── wsgi.py          # WSGI application
|   ├── supabase_client.py
|   └── supabase_adapter.py
├── usuarios/            # App de usuarios
|   ├── models/
|   |   ├── user_models.py
|   |   ├── notification_models.py
|   |   ├── audit_models.py
|   |   └── invitation_models.py
|   ├── serializers/
|   ├── views/
|   |   ├── user_views.py
|   |   ├── admin_views.py
|   |   ├── invitation_views.py
|   |   └── setup_views.py
|   ├── auth/
|   |   ├── backends.py
|   |   ├── authentication.py
|   |   └── permissions.py
|   └── urls/
├── libros/              # App de biblioteca
|   ├── models.py
|   ├── views.py
|   ├── serializers.py
|   ├── urls.py
|   └── google_books_service.py
├── manage.py
└── requirements.txt
```

4.2. Modelos de Datos y Base de Datos

La base de datos principal es **PostgreSQL** alojada en **Supabase**. Los modelos de Django definen la estructura de la base de datos.

El sistema se centra en las siguientes entidades principales y sus relaciones:



Modelos Clave

- **Usuario (App usuarios):** Hereda de `AbstractBaseUser` de Django. Incluye campos específicos como `rol` (`administrador`, `docente`, `estudiante`), `supabase_id` (clave de sincronización con Supabase Auth), y métricas de gestión de préstamos (`prestamos_activos`, `multas_pendientes`).
- **Libro (App libros):** Contiene metadatos del libro (`titulo`, `autor`, `isbn`) y campos de inventario (`cantidad_total`, `cantidad_disponible`, `estado`).
- **Prestamo (App libros):** Registra la relación entre un `Usuario` y un `Libro`, con fechas de préstamo y devolución, y un campo para el estado del préstamo.

El sistema utiliza un **esquema de autenticación híbrido** para aprovechar la gestión de usuarios de Supabase y la flexibilidad de Django/DRF.

1. **Registro/Login:** El *frontend* interactúa directamente con **Supabase Auth** para el registro y el inicio de sesión.
2. **Sincronización:** Tras el login exitoso en Supabase, el *backend* de Django utiliza el `supabase_id` para sincronizar o crear el usuario en su propia base de datos.
3. **Autorización API:** El *backend* utiliza `djangoRESTframework-simplejwt` para generar *tokens* JWT que son usados para autorizar las solicitudes a la API.

- **Permisos:** Se implementan permisos personalizados (**IsAdministrador**, **IsDocente**) para restringir el acceso a *endpoints* sensibles (ej. creación de libros, aprobación de préstamos).
-

5. Arquitectura del Frontend (edubooks-app)

El *frontend* es una aplicación **Angular** que utiliza **ionic** para los componentes de UI y **Capacitor** para el *build* nativo.

5.1. Estructura de Carpetas

El proyecto sigue la convención de Angular/Ionic:

```
dubooks-app/
├─ src/
│   └─ app/
│       └─ core/
│           └─ guards/
│               ├── auth-guard.ts
│               ├── no-auth-guard.ts
│               ├── role.guard.ts
│               ├── setup.guard.ts
│               └─ system-init.guard.ts
│           └─ interceptors/
│               ├── auth.interceptor.spec.ts
│               └─ auth.interceptor.ts
│           └─ models/
│               ├── libro.model.ts
│               └─ usuario.model.ts
│           └─ services/
│               ├── api.service.spec.ts
│               ├── api.service.ts
│               ├── auth.service.spec.ts
│               └─ auth.service.ts
```

```
| | | └─ biblioteca.service.spec.ts
| | | └─ biblioteca.service.ts
| | | └─ focus-management.service.ts
| | | └─ google-books.service.ts
| | | └─ lock-manager.service.ts
| | | └─ notificaciones.service.ts
| | | └─ permission.service.ts
| | | └─ sancion.service.ts
| | | └─ setup.service.ts
| | | └─ supabase.service.ts
| | | └─ usuario.service.ts
| | └─ pages/
| | | └─ administrador/
| | | | └─ admin-invitaciones/
| | | | | └─ admin-invitaciones-routing.module.ts
| | | | | └─ admin-invitaciones.module.ts
| | | | | └─ admin-invitaciones.page.html
| | | | | └─ admin-invitaciones.page.scss
| | | | | └─ admin-invitaciones.page.spec.ts
| | | | | └─ admin-invitaciones.page.ts
| | | | └─ admin-libros/
| | | | | └─ libro-form/
| | | | | | └─ libro-preview/
| | | | | | | └─ libro-preview.page.html
| | | | | | | └─ libro-preview.page.scss
| | | | | | | └─ libro-preview.page.ts
| | | | | | └─ libro-form.modal.html
| | | | | | └─ libro-form.modal.scss
| | | | | | └─ libro-form.modal.ts
| | | | | └─ admin-libros-routing.module.ts
| | | | | └─ admin-libros.module.ts
| | | | | └─ admin-libros.page.html
| | | | | └─ admin-libros.page.scss
```

```
| | | | | └─ admin-libros.page.spec.ts
| | | | | └─ admin-libros.page.ts
| | | | | └─ admin-prestamos/
| | | | | └─ admin-prestamos-routing.module.ts
| | | | | └─ admin-prestamos.module.ts
| | | | | └─ admin-prestamos.page.html
| | | | | └─ admin-prestamos.page.scss
| | | | | └─ admin-prestamos.page.spec.ts
| | | | | └─ admin-prestamos.page.ts
| | | | | └─ admin-sanciones/
| | | | | └─ crear-sancion/
| | | | | | └─ crear-sancion.page.html
| | | | | | └─ crear-sancion.page.scss
| | | | | | └─ crear-sancion.page.ts
| | | | | └─ admin-sanciones-routing.module.ts
| | | | | └─ admin-sanciones.module.ts
| | | | | └─ admin-sanciones.page.html
| | | | | └─ admin-sanciones.page.scss
| | | | | └─ admin-sanciones.page.ts
| | | | | └─ admin-solicitudes/
| | | | | └─ admin-solicitudes-routing.module.ts
| | | | | └─ admin-solicitudes.module.ts
| | | | | └─ admin-solicitudes.page.html
| | | | | └─ admin-solicitudes.page.scss
| | | | | └─ admin-solicitudes.page.spec.ts
| | | | | └─ admin-solicitudes.page.ts
| | | | | └─ admin-usuarios/
| | | | | └─ admin-usuarios-routing.module.ts
| | | | | └─ admin-usuarios.module.ts
| | | | | └─ admin-usuarios.page.html
| | | | | └─ admin-usuarios.page.scss
| | | | | └─ admin-usuarios.page.spec.ts
| | | | | └─ admin-usuarios.page.ts
```

```
| | | └─ ajustes/
| | | | └─ ayuda/
| | | | | └─ ayuda-routing.module.ts
| | | | | └─ ayuda.module.ts
| | | | | └─ ayuda.page.html
| | | | | └─ ayuda.page.scss
| | | | | └─ ayuda.page.ts
| | | | └─ comentarios/
| | | | | └─ comentarios-routing.module.ts
| | | | | └─ comentarios.module.ts
| | | | | └─ comentarios.page.html
| | | | | └─ comentarios.page.scss
| | | | | └─ comentarios.page.ts
| | | | └─ editar-perfil/
| | | | | └─ editar-perfil-routing.module.ts
| | | | | └─ editar-perfil.module.ts
| | | | | └─ editar-perfil.page.html
| | | | | └─ editar-perfil.page.scss
| | | | | └─ editar-perfil.page.ts
| | | └─ catalogo/
| | | | └─ catalogo-routing.module.ts
| | | | └─ catalogo.module.ts
| | | | └─ catalogo.page.html
| | | | └─ catalogo.page.scss
| | | | └─ catalogo.page.spec.ts
| | | | └─ catalogo.page.ts
| | | └─ detalle-libro/
| | | | └─ detalle-libro-routing.module.ts
| | | | └─ detalle-libro.module.ts
| | | | └─ detalle-libro.page.html
| | | | └─ detalle-libro.page.scss
| | | | └─ detalle-libro.page.spec.ts
| | | | └─ detalle-libro.page.ts
```

```
| | | └─ docente_y_estudiante/
| | | | └─ bibliografia/
| | | | | └─ bibliografia-routing.module.ts
| | | | | └─ bibliografia.module.ts
| | | | | └─ bibliografia.page.html
| | | | | └─ bibliografia.page.scss
| | | | | └─ bibliografia.page.spec.ts
| | | | | └─ bibliografia.page.ts
| | | | └─ bibliografia-estudiante/
| | | | | └─ bibliografia-estudiante-routing.module.ts
| | | | | └─ bibliografia-estudiante.module.ts
| | | | | └─ bibliografia-estudiante.page.html
| | | | | └─ bibliografia-estudiante.page.scss
| | | | | └─ bibliografia-estudiante.page.spec.ts
| | | | | └─ bibliografia-estudiante.page.ts
| | | | └─ historial-prestamos/
| | | | | └─ historial-prestamos-routing.module.ts
| | | | | └─ historial-prestamos.module.ts
| | | | | └─ historial-prestamos.page.html
| | | | | └─ historial-prestamos.page.scss
| | | | | └─ historial-prestamos.page.spec.ts
| | | | | └─ historial-prestamos.page.ts
| | | | └─ mis-solicitudes/
| | | | | └─ mis-solicitudes-routing.module.ts
| | | | | └─ mis-solicitudes.module.ts
| | | | | └─ mis-solicitudes.page.html
| | | | | └─ mis-solicitudes.page.scss
| | | | | └─ mis-solicitudes.page.spec.ts
| | | | | └─ mis-solicitudes.page.ts
| | | | └─ sanciones/
| | | | | └─ sanciones-routing.module.ts
| | | | | └─ sanciones.module.ts
| | | | | └─ sanciones.page.html
```

```
| | | | └─ sanciones.page.scss
| | | | └─ sanciones.page.spec.ts
| | | | └─ sanciones.page.ts
| | | └─ home/
| | |   └─ home-routing.module.ts
| | |   └─ home.module.ts
| | |   └─ home.page.html
| | |   └─ home.page.scss
| | |   └─ home.page.spec.ts
| | |   └─ home.page.ts
| | └─ app-routing.module.ts
| | └─ app.component.html
| | └─ app.component.scss
| | └─ app.component.spec.ts
| | └─ app.component.ts
| | └─ app.module.ts
| └─ environments/
| └─ assets/
└─ android/
  └─ app/...
  └─ gradle/...
  └─ build.gradle
  └─ capacitor.settings.gradle
  └─ gradle.properties
  └─ gradlew
  └─ gradlew.bat
  └─ settings.gradle
  └─ variables.gradle
└─ capacitor.config.ts
└─ ionic.config.json
└─ angular.json
└─ karma.conf.js
└─ .eslintrc.json
```

```
├─ .editorconfig
├─ .browserslistrc
├─ .gitignore
├─ package.json
└─ package-lock.json
```

5.2. Flujo de Datos y Servicios

El flujo de datos sigue el patrón **MVVM (Model-View-ViewModel)** implícito en Angular:

1. **Vistas (Pages):** Las páginas de Ionic (ej. `catalogo.page.ts`) llaman a los métodos de los servicios.
2. **Servicios (Services):** Los servicios (ej. `LibrosService`) contienen la lógica de negocio del *frontend* y utilizan el `HttpClient` de Angular para comunicarse con la API REST del *backend*.
3. **Modelos (Models):** Las interfaces TypeScript en `core/models/` definen la estructura de los datos recibidos del *backend*, asegurando la coherencia de tipos.
4. **Interceptors:** El `AuthInterceptor` se encarga de adjuntar el *token* JWT a cada solicitud saliente a la API de Django, y maneja la expiración del *token*.

6. API REST - Endpoints

La API REST se implementa con **Django REST Framework** y sigue convenciones RESTful.

6.1. Módulos de la API

La API se divide en módulos principales, accesibles bajo el prefijo `/api/`:

Módulo	Prefijo URL	Funcionalidades Clave
Usuarios	<code>/api/usuarios/</code>	Registro, Login, Perfil, Notificaciones, Gestión de Docentes/Estudiantes.
Libros	<code>/api/libros/</code>	CRUD de Libros, Búsqueda, Categorías.
Préstamos	<code>/api/libros/prestamos/</code>	Solicitud, Aprobación, Devolución, Renovación.

Módulo	Prefijo URL	Funcionalidades Clave
Reservas	/api/libros/reservas/	Creación y Cancelación de Reservas.
Sanciones	/api/libros/sanciones/	Listado y Pago de Sanciones.
Estadísticas	/api/libros/estadisticas/	Reportes administrativos (ej. préstamos vencidos).

6.2. Ejemplo de Endpoint Detallado

Endpoint: `POST /api/libros/prestamos/`

Parámetro	Valor	Descripción
Método	<code>POST</code>	Creación de un nuevo recurso (solicitud de préstamo).
Permisos	<code>IsAuthenticated</code> (Usuario Docente o Estudiante)	Requiere autenticación JWT.
Cuerpo (Request Body)	<code>{"libro_id": 123, "fecha_devolucion_esperada": "YYYY-MM-DD"}</code>	ID del libro a solicitar y fecha de devolución propuesta.
Respuesta Exitosa (201 Created)	<code>{"id": 456, "libro": "...", "usuario": "...", "estado": "Pendiente"}</code>	Objeto del préstamo creado.
Errores Comunes	<code>400 Bad Request</code>	Libro no disponible, usuario con límite de préstamos excedido.
	<code>401 Unauthorized</code>	Token JWT inválido o ausente.

7. Infraestructura y Despliegue

El despliegue del *backend* se realiza en **Render**, mientras que la base de datos y la autenticación se gestionan a través de **Supabase**.

7.1. Entornos de Despliegue

Se recomienda la siguiente configuración de entornos:

Entorno	Propósito	Backend	Base de Datos
Desarrollo (Local)	Desarrollo y pruebas unitarias.	<code>python manage.py runserver</code>	SQLite o Supabase local/staging.
Staging	Pruebas de QA y pre-producción.	Render Web Service (Rama <code>staging</code>)	Supabase Staging DB.
Producción	Entorno en vivo.	Render Web Service (Rama <code>main</code>)	Supabase Production DB.

7.2. Configuración de Render (Backend)

El *backend* de Django se despliega como un **Render Web Service**.

- **Comando de Build:** `pip install -r requirements.txt && python manage.py collectstatic --noinput`
- **Comando de Inicio:** `gunicorn edubooks.wsgi:application --bind 0.0.0.0:$PORT --workers 4`
- **CI/CD:** Se asume una configuración de **Integración Continua/Despliegue Continuo (CI/CD)** donde un *push* a la rama `main` o `staging` dispara automáticamente el *build* y el despliegue en Render.

7.3. Variables de Entorno Críticas

Las variables de entorno se gestionan mediante el archivo `.env` en desarrollo y a través de la configuración de **Environment Variables** en Render para producción.

Variable	Componente	Descripción
<code>SECRET_KEY</code>	Backend	Clave secreta de Django. Crítica para la seguridad.

Variable	Componente	Descripción
DEBUG	Backend	<code>True</code> para desarrollo, <code>False</code> para producción.
ALLOWED_HOSTS	Backend	Dominios permitidos (incluye el dominio de Render).
SUPABASE_URL	Ambos	URL de la instancia de Supabase.
SUPABASE_ANON_KEY	Ambos	Clave pública de Supabase (para uso en el <i>frontend</i>).
SUPABASE_SERVICE_KEY	Backend	Clave de servicio de Supabase (para uso en el <i>backend</i>). Crítica para la seguridad.
DB_ENGINE, DB_HOST, etc.	Backend	Credenciales de conexión a la base de datos PostgreSQL.

8. Instalación y Configuración Local

8.1. Requisitos Previos

- **Git**
- **Python 3.11+**
- **Node.js 18+**
- **npm o yarn**
- **Cuenta Supabase** (para obtener credenciales de DB y Auth).

8.2. Instalación del Backend

1. Clonar el repositorio

git clone <https://github.com/Mauricio50-byte/Edubooks.git> cd Edubooks/edubooks-api

2. Crear y activar entorno virtual

```
python3 -m venv venv source venv/bin/activate
```

3. Instalar dependencias

```
pip install -r requirements.txt
```

4. Configurar variables de entorno

```
cp .env.example .env
```

Editar el archivo `.env` con las credenciales de Supabase y la `SECRET_KEY`

5. Ejecutar migraciones

```
python manage.py migrate
```

6. Crear superusuario (Administrador)

```
python manage.py createsuperuser
```

7. Iniciar el servidor de desarrollo

```
python manage.py runserver
```

El servidor estará disponible en <http://127.0.0.1:8000/>

8.3. Instalación del Frontend

1. Navegar al directorio del frontend

```
cd ../edubooks-app
```

2. Instalar dependencias de Node.js

```
npm install
```

3. Configurar environment

Editar `src/environments/environment.ts` con la URL de la API (ej. <http://127.0.0.1:8000/api>)

y las credenciales de Supabase.

4. Iniciar el servidor de desarrollo de Ionic

`ionic serve`

La aplicación se abrirá en el navegador (ej. <http://localhost:8100/>)

9. Mantenimiento, Logs y Seguridad

9.1. Gestión de Errores y Logs

- **Backend:** Django utiliza el sistema de *logging* estándar de Python. En producción (Render), los logs se envían a la salida estándar (`stdout/stderr`) y son capturados por el servicio de *logging* de Render.
 - **Errores de API:** Los errores de validación y negocio se devuelven con códigos de estado HTTP apropiados (ej. `400 Bad Request`, `404 Not Found`, `403 Forbidden`).
- **Frontend:** Los errores se capturan mediante el sistema de manejo de errores de Angular y se pueden integrar con servicios de monitoreo de errores (ej. Sentry, Crashlytics) para el seguimiento en producción.

9.2. Consideraciones de Seguridad

- **Autenticación Híbrida:** La delegación de la gestión de contraseñas a Supabase Auth reduce la superficie de ataque del *backend* de Django.
- **Tokens JWT:** Se utiliza `django-rest-framework-simplejwt` con tiempos de expiración definidos (`ACCESS_TOKEN_LIFETIME`: 60 minutos) y rotación de *refresh tokens*.
- **Validación de Datos:** El *backend* implementa validación de datos a nivel de *serializer* y modelo (ej. validadores de rango de año en `Libro`), previniendo inyecciones y datos inconsistentes.

- **Variables de Entorno:** Las claves sensibles (`SECRET_KEY`, `SUPABASE_SERVICE_KEY`) **NUNCA** deben estar en el código fuente y deben ser gestionadas de forma segura por el entorno de despliegue (Render).

9.3. Requerimientos para Mantenimiento y Evolución Futura

1. Monitoreo de Rendimiento y Observabilidad:

- **Implementación de APM:** Integrar una solución de **Application Performance Monitoring (APM)** como **New Relic** o **Datadog** para obtener métricas detalladas sobre la latencia, el rendimiento de las consultas a la base de datos y el uso de recursos en los *endpoints* críticos de Django. Esto es crucial para la escalabilidad.
- **Alertas Proactivas:** Configurar alertas basadas en umbrales de error y rendimiento para garantizar la **observabilidad** del sistema en producción.

2. Robustez y Cobertura de Pruebas:

- **Ampliación de Pruebas:** Aumentar significativamente la cobertura de **pruebas unitarias y de integración**, con un enfoque particular en la lógica de negocio compleja del módulo `libros` (cálculo de sanciones, gestión de estados de préstamos, validación de reservas).
- **Pruebas de Carga:** Implementar pruebas de carga (ej. con Locust o JMeter) para simular el tráfico de usuarios y asegurar que el *backend* en Render pueda manejar picos de demanda.

3. Documentación de API Automatizada:

- **Generación Automática:** Integrar una herramienta como **drf-spectacular** en el *backend* de Django para generar automáticamente la documentación de la API en formato **OpenAPI/Swagger**. Esto asegura que la documentación esté siempre sincronizada con el código, facilitando la integración del *frontend* y de terceros.

4. Evolución del Módulo de Sanciones (Pagos Reales):

- **Integración de Pasarela de Pago:** Desarrollar la funcionalidad para que las sanciones generadas puedan ser pagadas a través de una **pasarela de pago real** (ej. Stripe, PayPal, o un proveedor local).
- **Flujo de Pago:** Implementar el flujo completo: creación de la orden de pago, redirección al proveedor, manejo de *webhooks* de confirmación de pago, y actualización del estado de la sanción y del usuario (`multas_pendientes`, `sancionado_hasta`) en la base de datos.

5. Notificaciones Proactivas (Push):

- **Integración de FCM/APNS:** Integrar un servicio de notificaciones *push* (ej. **Firebase Cloud Messaging - FCM**) a través de **Capacitor** para enviar alertas críticas a los usuarios, como recordatorios de devolución de libros o notificaciones de préstamos vencidos.

6. Optimización de Consultas a Base de Datos:

- **Análisis de ORM:** Realizar una auditoría de las consultas generadas por el ORM de Django para identificar y optimizar consultas N+1 o ineficientes, especialmente en las vistas de listado de préstamos y libros.
- **Indexación:** Revisar y asegurar la correcta indexación de campos clave utilizados en búsquedas y relaciones (*isbn*, *usuario_id*, *libro_id*).

Fin del Documento